MealIQ - Stage 2 Implementation Report

EECS3311 Software Design, York University. Repository: https://github.com/Ayech14/MealIQ

MealIQ is a planning aid, not medical or dietary advice.

1. Project overview and implementation status

MealIQ is an AI agent for meal planning and grocery optimization. It serves a home cook who has to balance nutrition goals, allergies and dietary rules, food preferences, what is already in the pantry, a weekly grocery budget, and how much time each day allows for cooking.

The application is written in Java 21 with Maven, as the instructor required on October 30, 2026. It replaces a first Python implementation started in Period 1 (see Section 4, C01).

Interfaces. Both interfaces call the same `MealPlanningFacade` in-process:

- **GUI:** a JavaFX desktop app (`./mealiq-gui`) with a sidebar of pages. It opens on Home, where the agent's check-up proposes the next step, followed by Ask the agent, Meal plan, Pantry, Recipes, Groceries and Profile.
- **CLI:** a picocli command line (`./mealiq`). Commands: `ask`, `profile`, `pantry`, `recipes`, `plan`, `grocery`, `ingredients`, `setup`.

AI integration.

- The agent uses OpenAI `gpt-5.4-mini-2026-03-17` through the OpenAI Java SDK's Responses API, with strict JSON-schema output, behind the provider-neutral `LLMClient` interface.
- The model is pinned in the local `.env` file.

Agent behaviour.

- **Tool selection:** the LLM chooses tools in a loop bounded by `MAX_TOOL_STEPS`. The tools are `recipeSearch`, `nutritionAnalysis`, `groceryList` and `substituteIngredient`. `ToolManager` rejects invalid calls and returns the error to the LLM as its next observation.
- **Ranking:** deterministic planning strategies rank the candidates, and the LLM chooses among them and explains the choice.
- **Memory:** the agent keeps a short memory of earlier decisions.
- **What the LLM never does:** it never calculates nutrition, prices or quantities, and never changes stored data. Every proposal is validated deterministically, previewed, and applied only after the user confirms, through a Command that can be undone.

Data.

- **Nutrition:** USDA FoodData Central SR Legacy, curated to 146 ingredients.
- **Prices:** Statistics Canada average retail prices (table 18-10-0245-01, August 2026) for 63 of those ingredients.
- **Recipes:** 30 curated seed recipes built only from catalogue ingredients.
- **Storage:** PostgreSQL through hand-written JDBC repositories, with H2 in PostgreSQL mode for tests.

Working functionality. All 13 Stage 1 features (F01-F13) and two new features (N01, the Assistant; N02, the agent check-up on Home) work in the CLI and the GUI. Every AI feature was run against the real OpenAI API and a real PostgreSQL database during development (Section 6).

Limitations and incomplete parts:

- **No external recipe provider.** `ExternalRecipeAdapter` from Stage 1 is not implemented: recipe search covers the 30 curated recipes plus recipes the AI generates and the user saves (C13).

- **Partial prices.** About half the catalogue has no Statistics Canada price, so cost estimates are lower bounds shown as `$x+`. The budget is enforced only when known prices alone exceed it (C17).
- **Portions of one or two servings.** Portions are sized deterministically up to two servings per meal (C32), so a very high calorie target can still be missed on some days. This is reported as a warning, never hidden.
- **Plan generation can fail on tight constraints.** The agent now sees, for each time-limited day, only the recipes that fit (C33), but with very few matching recipes a plan can still fail validation after its one retry. Nothing is saved and every violation is listed.
- **Untested GUI handlers.** JavaFX event handlers have no automated tests. The facade operations they call are tested, and every screen was rendered and inspected (Section 6).
- **Launch scripts.** `./mealiq` and `./mealiq-gui` are shell scripts for macOS and Linux. On Windows, the README shows the equivalent `java` command.

2. Feature to implementation mapping

Package prefix: `io.github.ayech14.mealiq`. Every feature is reached through `application/MealPlanningFacade`; the CLI commands are in `presentation/cli`, and the GUI screens are in `presentation/gui`.

ID and name	Stage 1	Status	Implemented classes/files and responsibilities	Key methods	Patterns	Type	Video
F01 User goals and preferences	F01	Implemented	`ProfileService` validates ranges and contradictions, saves, publishes changes; `UserProfile`; `DietRules` (supported diets, known allergens); `JdbcProfileRepository`	`updateProfile`, `validate`, `preferenceWarnings`, `publish`	Facade, Observer	Deterministic	pending
F02 Pantry inventory management	F02	Implemented	`PantryService` matches everyday names to catalogue ingredients, builds and stores items, publishes `PANTRY_CHANGED`; `Pantry` (`availableQuantity`, `shortfall`, `take`); `JdbcPantryRepository`	`matchIngredients`, `makeItem`, `updateItem`, `removeItem`, `expiredItems`	Facade, Observer	Deterministic	pending
F03 Recipe search and recommendation	F03	Modified (C13)	`AgentController.recommendRecipe` builds the context; `MealPlanningAgent.recommendRecipes` runs the tool loop; `ToolManager`, `RecipeTool`; `PlanningStrategy` ranks; `ConstraintValidator` re-checks every result	`recommendRecipes`, `runToolLoop`, `rankRecipes`, `validateRestrictions`	Strategy, Factory Method, Adapter, Facade	Hybrid	pending
F04 AI recipe generation	F04	Implemented	`MealPlanningAgent.generateRecipe` with a schema listing only allowed catalogue ingredients; `PromptBuilder.buildRecipePrompt`; deterministic checks with one retry; `AgentController` re-validates; saved as `GENERATED`	`generateRecipe`, `recipeProblems`, `saveGeneratedRecipe`	Adapter, Facade	Hybrid	pending
F05 Weekly meal-plan generation	F05	Implemented	`MealPlanningAgent.createPlanProposal` gathers candidates per meal type (widening over-narrow searches), ranks them and lets the LLM fill slots from per-day lists; `PortionService.sizePortions`; `ProposalValidator` + `ConstraintValidator.validatePlan`; one retry with violations; `MealPlanService.createFromProposal`	`generateWeeklyPlan`, `createPlanProposal`, `sizePortions`, `validate`, `validatePlan`	Strategy, Factory Method, Facade, Adapter	Hybrid	pending
F06 Nutrition analysis	F06	Implemented	`NutritionService` scales per-100 g USDA values by grams; `NutritionTool` exposes it to the agent; daily totals shown against targets	`calculateNutrition`, `dailyTotals`, `analyze`	Facade	Deterministic	pending
F07 Grocery-list generation	F07	Implemented	`GroceryService.generateList` sums every meal's needs, combines units safely, subtracts usable pantry stock once; `GroceryTool`	`generateList`, `consolidate`, `refreshForPlan`, `project`	Facade	Deterministic	pending

ID and name	Stage 1	Status	Implemented classes/files and responsibilities	Key methods	Patterns	Type	Video
F08 Grocery-list optimization	F08	Implemented	`GroceryService.optimizeList` prices items with `StatCanPriceDataSource`; `BudgetService.evaluate` gives the shortfall and cost drivers; over budget leads to `AgentController.resolveBudgetConflict` (cheaper meals, previewed)	`optimizeList`, `evaluate`, `resolveBudgetConflict`, `proposeChange`	Adapter, Facade, Command	Hybrid	pending
F09 Ingredient substitution	F09	Implemented	`SubstitutionTool.findCandidates` filters catalogue substitutes by the profile; the LLM picks; `SubstituteIngredientCommand` applies a recipe variant	`requestSubstitution`, `proposeSubstitution`, `validateChange`, `execute`, `undo`	Command, Facade	Hybrid	pending
F10 Natural-language plan modification	F10	Implemented	`MealPlanningAgent.proposeChange` (replace, clarify or no change); `ProposalValidator` + `ConstraintValidator.validateChange`; held in `pendingChanges` until `confirmChange`; `MealPlanService.executeChange/undoLastChange`	`interpretModification`, `proposeChange`, `confirmChange`, `undoLastChange`	Command, Facade, Adapter	Hybrid	pending
F11 Cooking-time-aware planning	F11	Implemented	`TimeAwareStrategy` ranks by the day's limit; `ConstraintValidator.validateTime`; `UpdatePlanConstraintCommand` adapts a day to a new limit	`rank`, `validateTime`, `execute`	Strategy, Factory Method, Command	Hybrid	pending
F12 Budget-aware planning	F12	Implemented	`BudgetStrategy` ranks by known cost; `BudgetService.estimateCost` uses pantry stock once; plans over budget on known prices are rejected	`rank`, `estimateCost`, `evaluate`, `validatePlan`	Strategy, Factory Method	Hybrid	pending
F13 Dynamic plan adaptation	F13	Implemented	`MealPlanImpactObserver` and `GroceryListImpactObserver` react to published changes; `PlanImpactAssessor` finds affected meals; `MealPlanningAgent.proposeAdaptation` replaces only those	`publish`, `onPlanRelevantChange`, `assessImpact`, `recordImpact`, `getPendingImpact`, `adaptPlan`	Observer, Command, Facade	Hybrid	pending
N01 Assistant (typed requests)	New	New (C30)	`MealPlanningAgent.interpretRequest` maps a message to one of 12 actions with arguments; `MealPlanningFacade.ask` runs the matching use case; pantry and profile updates wait for `confirmUpdate`; `AssistantView`, `AskCommand`	`interpretRequest`, `ask`, `confirmUpdate`	Facade, Adapter	AI-based routing over hybrid features	pending
N02 Agent check-up (Home)	New	New (C35)	`MealPlanningFacade.checkup` looks at the profile, plan, pending impacts, pantry expiry and budget and returns `Checkup` with suggestions; each suggestion carries the request the agent runs if the user accepts; `HomeView`	`checkup`	Facade, Observer (reads recorded impacts)	Deterministic detection, AI-based action after confirmation	pending

How the classes collaborate.

- **F01:** The Profile tab or `profile set` builds a `UserProfile`. `MealPlanningFacade.updateProfile` calls `ProfileService.updateProfile`, which validates every field, rejects unsupported diets and unknown allergens (a hard constraint must never be silently ignored), saves through `ProfileRepository`, and publishes one `PlanRelevantChange` per kind of change (`PROFILE_CHANGED`, `BUDGET_CHANGED`, `TIME_LIMIT_CHANGED`).
- **F02:** `PantryService.makeItem` checks the ingredient against the catalogue and the unit and amount, and reports every bad field at once. `updateItem` saves and publishes `PANTRY_CHANGED`. Expired items stay listed but count as zero stock. Using the pantry before a profile exists creates a default profile first (`ensureProfile`).
- **F03:**
  1. `AgentController.recommendRecipe` builds a `PlanningContext` (profile, pantry, request text).
  2. In `runToolLoop`, the LLM answers with a `next_action` (tool name, arguments, reason).
  3. `ToolManager.execute` checks the call and runs `RecipeTool`, which never returns recipes that break the profile.
  4. The strategy chosen by `DefaultPlanningStrategyFactory` ranks the candidates.
  5. The LLM picks up to three and explains the choice, using only the facts in the prompt.
  6. The controller validates each pick again.
  7. Filters the user set explicitly (`--max-minutes`, for example) hold whatever arguments the LLM chose.
- **F04:**
  1. The recipe schema enumerates the catalogue ingredients the profile allows, so an allergen cannot be produced.
  2. `recipeProblems` checks what the schema cannot express: every amount must be weighable, and the meal type, time limit and required or excluded ingredients must match.
  3. A failing draft is retried once with its problems, then declined.
  4. The controller re-validates restrictions.
  5. The user previews the recipe before it is saved.
- **F05:**
  1. `createPlanProposal` lets the LLM search for each meal type, and searches automatically for any type it skipped.
  2. The strategy ranks candidates per type, and the LLM assigns them to slots (`MealSlots`).
  3. `ProposalValidator` checks structure: stored recipes, meal types, dates, each slot filled once, no recipe twice in a day.
  4. `ConstraintValidator.validatePlan` checks allergies, diet, per-day time and budget as limits, and nutrition targets as warnings.
  5. An invalid proposal is retried once with its violations. Only a valid plan is saved, as the profile's active plan.
- **F06:** `NutritionService.calculateNutrition` converts each ingredient quantity to grams with catalogue weights and scales its per-100 g USDA values. Missing data marks the summary incomplete instead of estimating (D09).
- **F07:** `GroceryService.generateList` scales each meal's ingredients to its servings. It combines units of the same ingredient by exact conversion, or by catalogue weight; otherwise the lines stay separate. It then subtracts pantry stock with `Pantry.take`, so stock is used once.
- **F08:** `optimizeList` prices each item to buy, through the `PriceDataSource` adapter. `BudgetService.evaluate` compares the total with the weekly budget scaled to the plan's length, and names the three biggest costs. When the plan is over budget, `plan cheaper` (or "Make cheaper", or the Assistant) asks the agent for cheaper replacements. They are previewed and applied as commands.
- **F09:**
  1. `SubstitutionTool.findCandidates` returns catalogue substitutes the profile allows and does not dislike.
  2. The LLM chooses one per ingredient.

3. The change becomes a recipe variant for every planned meal that uses the ingredient.
4. It is validated, previewed, and applied by `SubstituteIngredientCommand`, which undo reverses.

- **F10:**
  1. `proposeChange` lists the plan and the candidates the agent's searches found.
  2. The LLM answers `replace` (day, meal type, recipe), `clarify` (one question) or `no_change`.
  3. The change is validated as a whole, held as pending, and shown as a preview.
  4. `confirmChange` accepts only a change this controller proposed, once.
  5. `MealPlanService.executeChange` builds the commands, saves the plan, and stores the change for undo.
  6. The grocery list is refreshed.
- **F11:** `TimeAwareStrategy` ranks recipes against the limit for the plan's day, and over-limit recipes rank last. Plans and changes that exceed a day's limit are rejected. When a limit tightens, adaptation replaces only the meals over it, as `UpdatePlanConstraintCommand`.
- **F12:** `BudgetStrategy` scores recipes by known cost per serving, scaled by the share of priced ingredients, so unknown prices never make a recipe look cheap. `BudgetService.estimateCost` uses pantry stock once across the plan's meals. A plan or change whose known prices exceed the budget is rejected.
- **F13:**
  1. `PantryService` and `ProfileService` publish changes. The observers only do fast, deterministic work (D06): `GroceryListImpactObserver` marks the list stale, and `MealPlanImpactObserver` asks `MealPlanService.assessImpact` which meals now break a rule or use an ingredient that ran out, and records the cause.
  2. The interfaces show an impact notice.
  3. `adaptPlan` asks the agent for the smallest change, limited to the affected meals.
- **N01:**
  1. `interpretRequest` gives the LLM the user's situation (profile, pantry, plan, earlier decisions) and the list of actions.
  2. The answer is schema-constrained: the action is an enum, pantry ingredients must be catalogue names, and diets and allergens must be supported values.
  3. `MealPlanningFacade.ask` runs the matching use case with its normal checks.
  4. Pantry and profile updates are previewed and applied by `confirmUpdate`.
  5. When no stored recipe fits, generation is offered (UC03 step 5a).

3. Implemented design patterns

Six patterns from Stage 1 are implemented. All files are under `src/main/java/io/github/ayech14/mealiq/`.

Strategy (matches Stage 1).

- **Problem:** recipes must be ranked by different policies (nutrition goal, cooking time, cost) without the agent knowing how each one scores.
- **Roles:** `planning/PlanningStrategy` is the strategy interface (`configure`, `score`, `rank`). `NutritionGoalStrategy`, `TimeAwareStrategy` and `BudgetStrategy` are concrete strategies. `MealPlanningAgent.rankRecipes` is the client.
- **Why it helps:** each policy is a separate, unit-tested class (`PlanningStrategyTest`), and a new policy needs no change to the agent.
- **Difference from Stage 1:** `score` is public and `rank` is a default method, so the ordering rules (score, then time, then name) live in one place.

Factory Method (matches Stage 1).

- **Problem:** the agent needs the right strategy for a planning mode, configured with the user's profile, without depending on concrete classes.
- **Roles:** `planning/PlanningStrategyFactory` is the creator: `strategyFor(context)` calls the factory method `createStrategy(mode)`, then `configure(profile)`. `DefaultPlanningStrategyFactory` is the concrete creator. The strategies are the products.
- **Why it helps:** a new instance per request means strategies never share state.

Command (matches Stage 1; extended, see C24).

- **Problem:** plan changes proposed by the LLM must be separate from their effect, applied only after confirmation, and undoable.
- **Roles:**
  - `commands/PlanChangeCommand` is the command interface (`execute`, `undo`, `describe`, `swaps`).
  - `ReplaceMealCommand`, `SubstituteIngredientCommand` and `UpdatePlanConstraintCommand` are concrete commands. `CompositeCommand` is a macro command for changes that touch several meals.
  - `MealPlan` is the receiver (`replaceMeal`).
  - `MealPlanService` is the invoker: `executeChange`, `undoLastChange`, and `createCommand`, which builds commands from a `PlanChange`.
- **Why it helps:** the agent can only propose; the plan changes only through commands; each applied change can be undone.
- **Difference from Stage 1:** the swaps are stored in the database, so undo works across CLI sessions.

Observer (matches Stage 1).

- **Problem:** pantry and profile changes must flag the affected plan and grocery list without the services knowing about plans or the agent.
- **Roles:**
  - `events/PlanRelevantChangePublisher` is the subject interface, implemented by `PantryService` and `ProfileService`.
  - `PlanImpactObserver` is the observer interface. `MealPlanImpactObserver` and `GroceryListImpactObserver` are concrete observers.
  - `ObserverList` handles notification and isolates a failing observer.
  - Subscription happens in `MealPlanningFacade.subscribeObservers`.
- **Why it helps:** observers do only deterministic work, so updates stay fast and never call the LLM (D06).

Facade (matches Stage 1; the API layer was removed, see C02).

- **Problem:** two interfaces (GUI and CLI) must reach the same use cases with identical rules.
- **Roles:** `application/MealPlanningFacade` is the facade. The subsystem is `AgentController`, the services and the repositories. The clients are `MealPlanningCLI` and its commands, and `MealPlanningGUI` and its views.
- **Why it helps:** no business logic exists in the interfaces, and both give the same results.

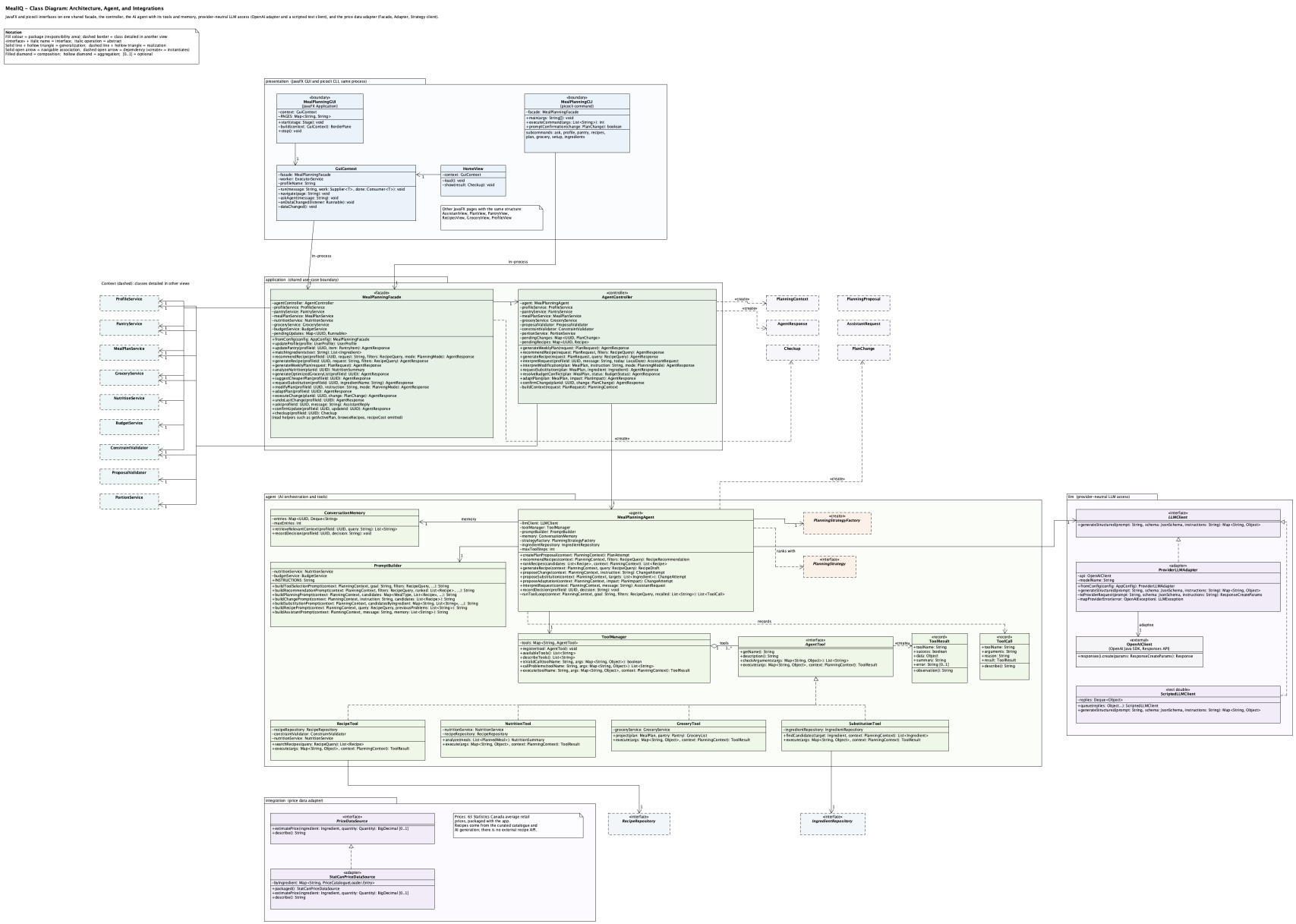
Adapter (matches Stage 1 for the LLM and prices; no recipe adapter, see C13).

- **Problem:** domain code must not depend on a provider SDK or a data file format.
- **Roles:**
  - `llm/LLMClient` is the target. `ProviderLLMAdapter` is the adapter, and the OpenAI Java SDK client is the adaptee. SDK types, errors and refusals are mapped to `LLMException` kinds. `ScriptedLLMClient` is a test implementation of the same target.
  - `integration/PriceDataSource` is the target, and `StatCanPriceDataSource` adapts the Statistics Canada price table, converting units with catalogue weights.

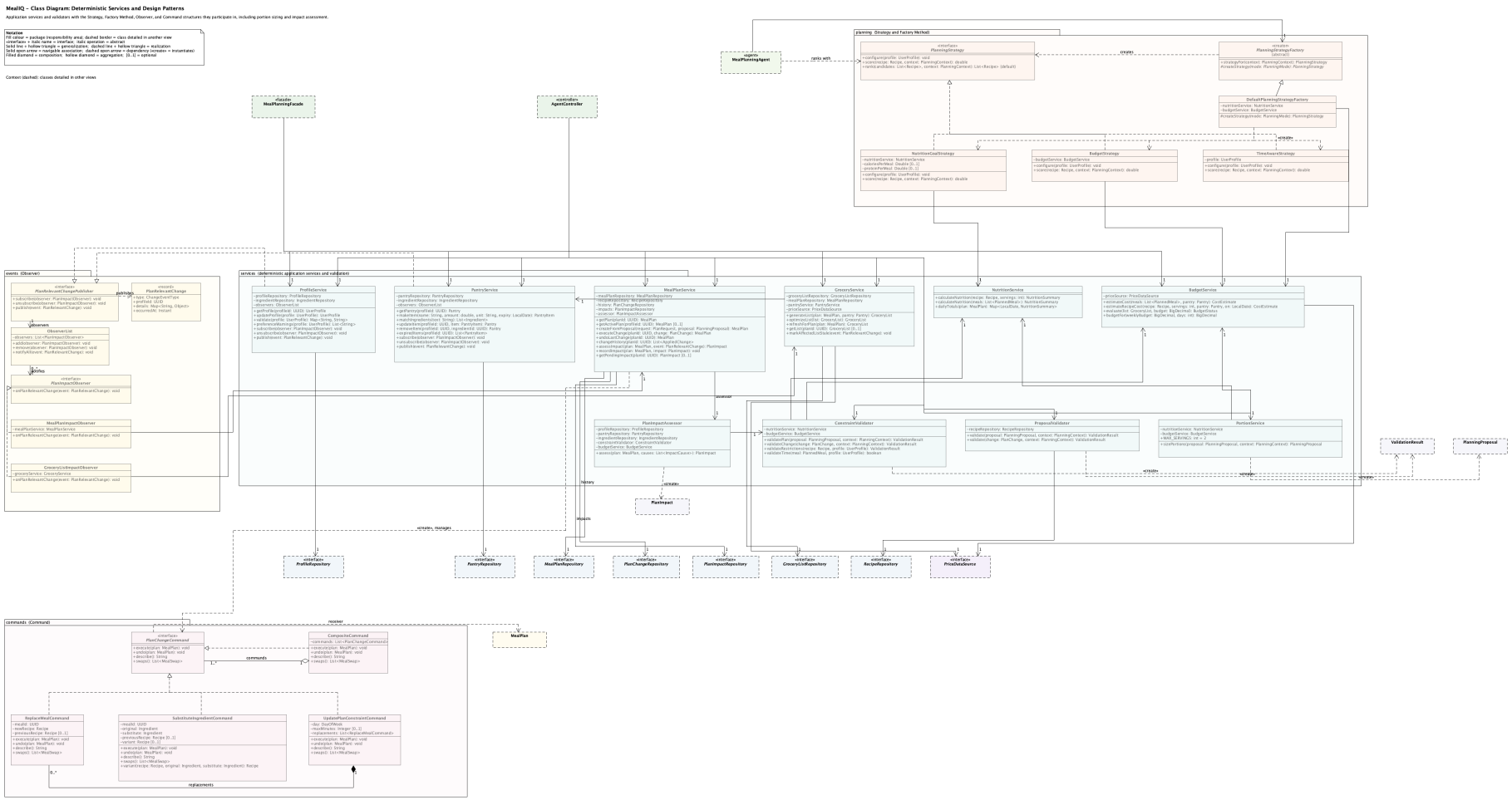
Updated UML diagrams

The use-case, class and sequence diagrams were updated to the Java implementation. They show JavaFX and picocli on the facade with no HTTP layer, the OpenAI adapter, the Statistics Canada price adapter, `PortionService`, `PlanImpactAssessor`, `CompositeCommand`, the assistant and check-up types, the two history repositories, and the new features N01 and N02. The UMLet sources and full-size PNG exports are in `docs/stage1/diagrams`; `class-diagram-complete.png` shows all 94 classes in one model.

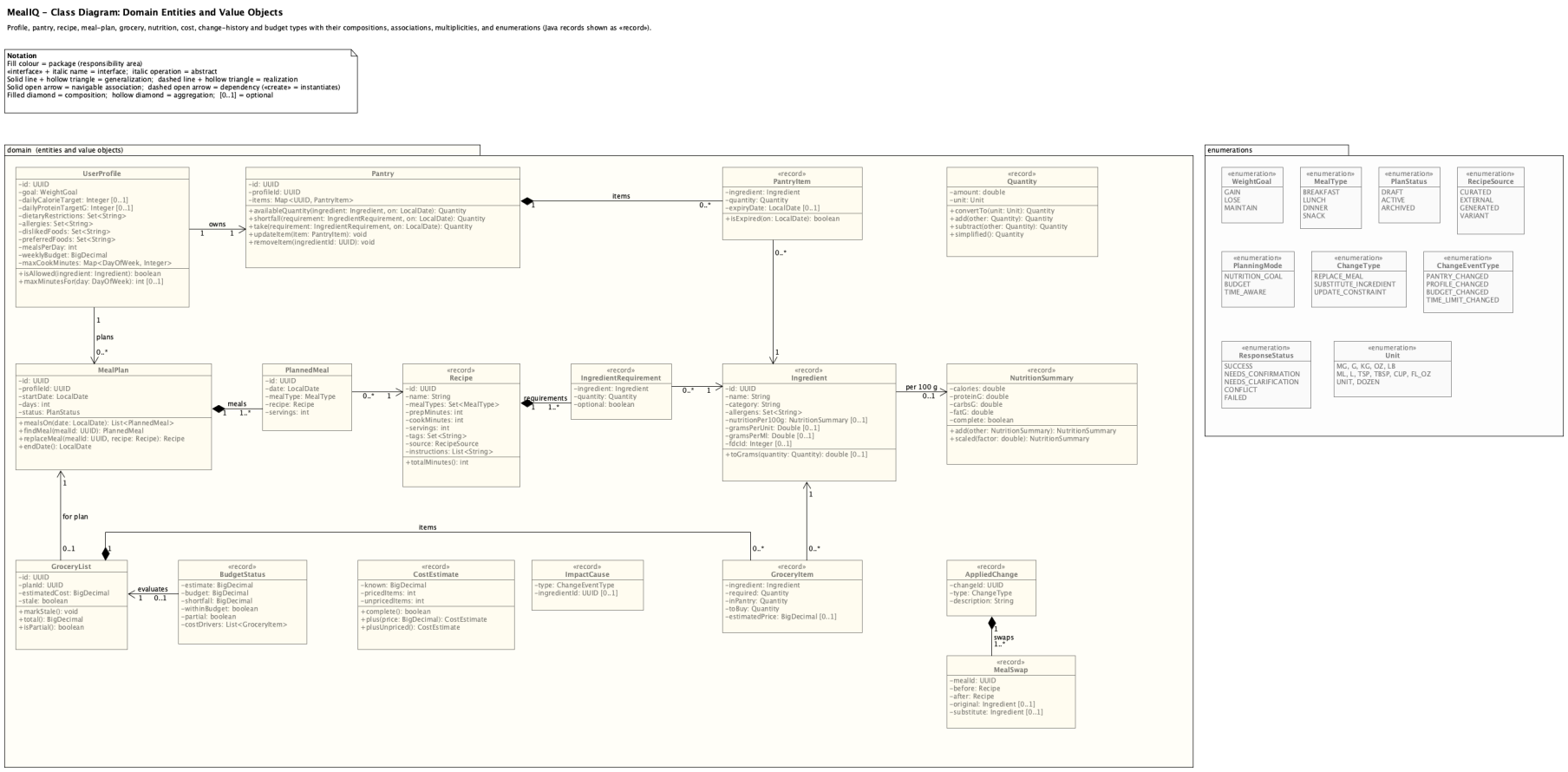
Class diagram: architecture, agent and integrations.



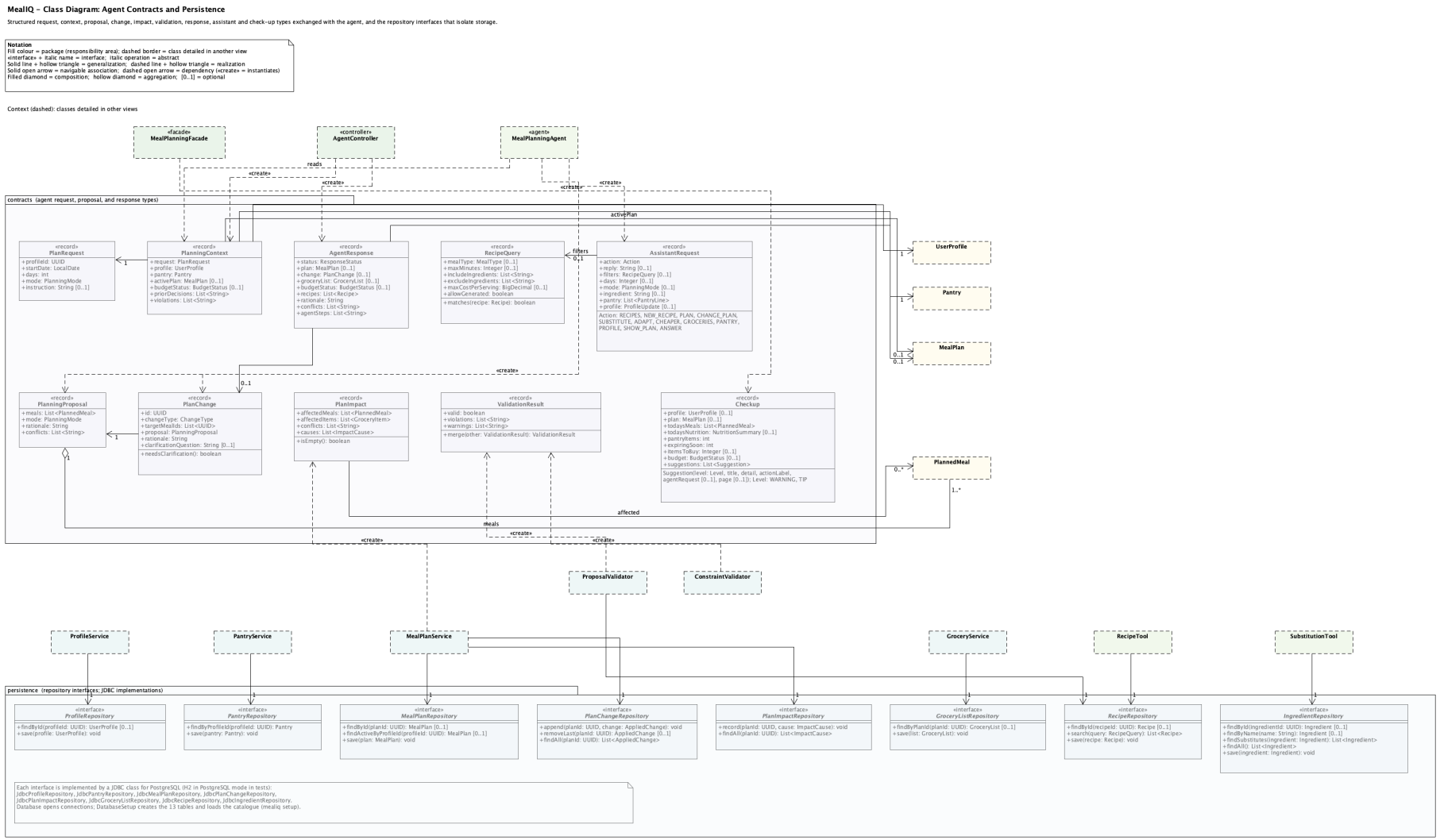
Class diagram: deterministic services and design patterns.



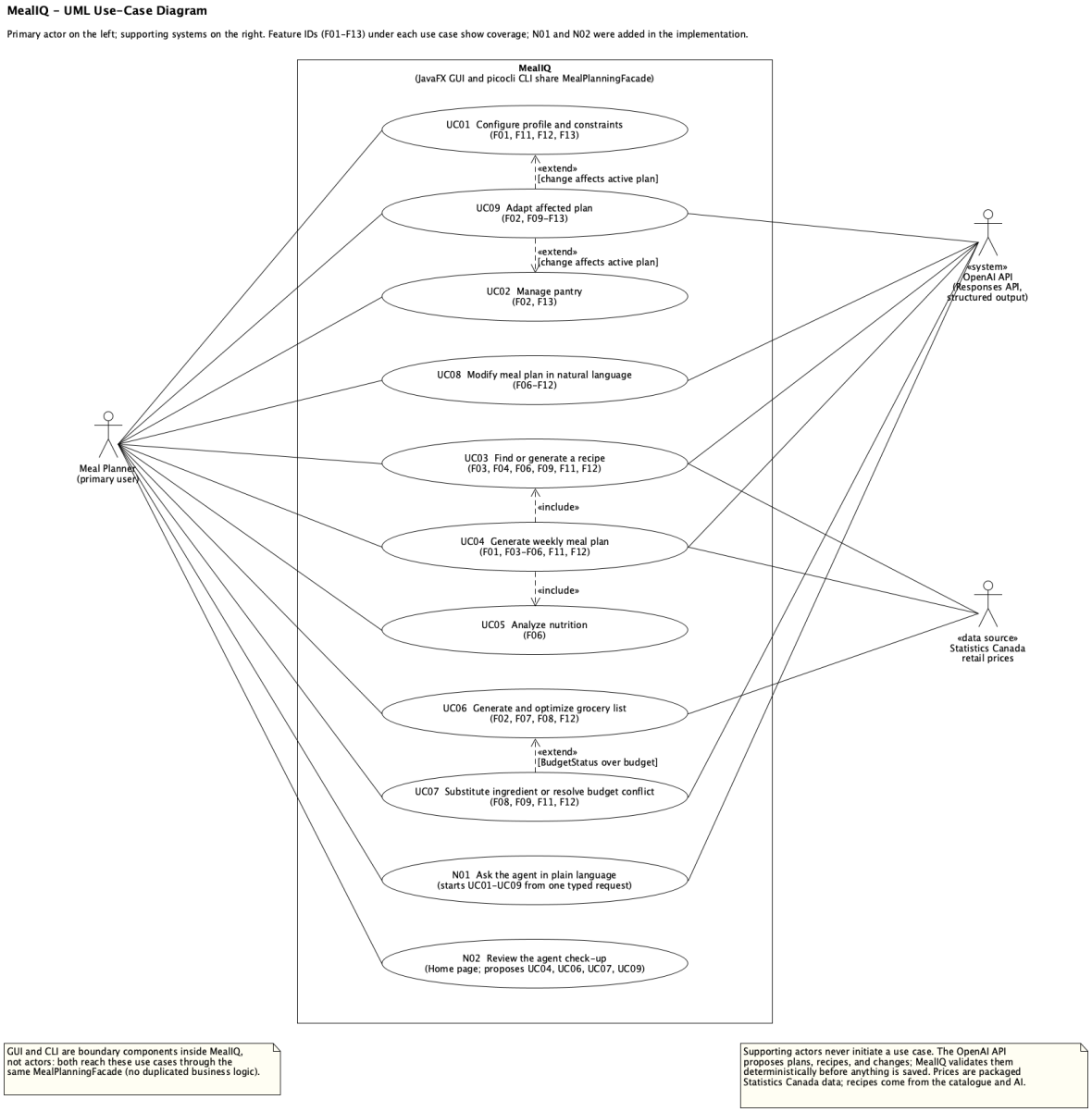
Class diagram: domain entities and value objects.



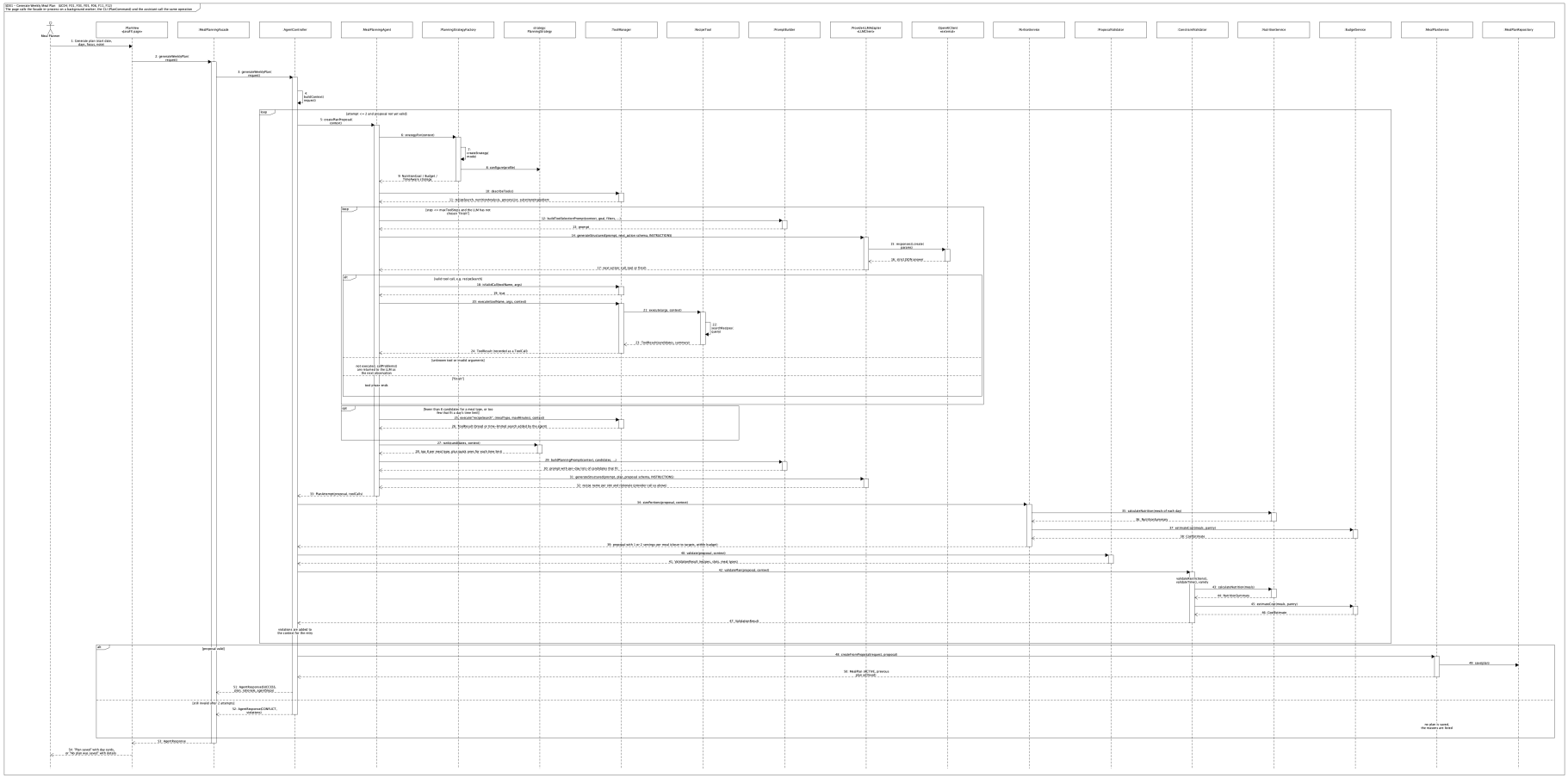
Class diagram: agent contracts and persistence.



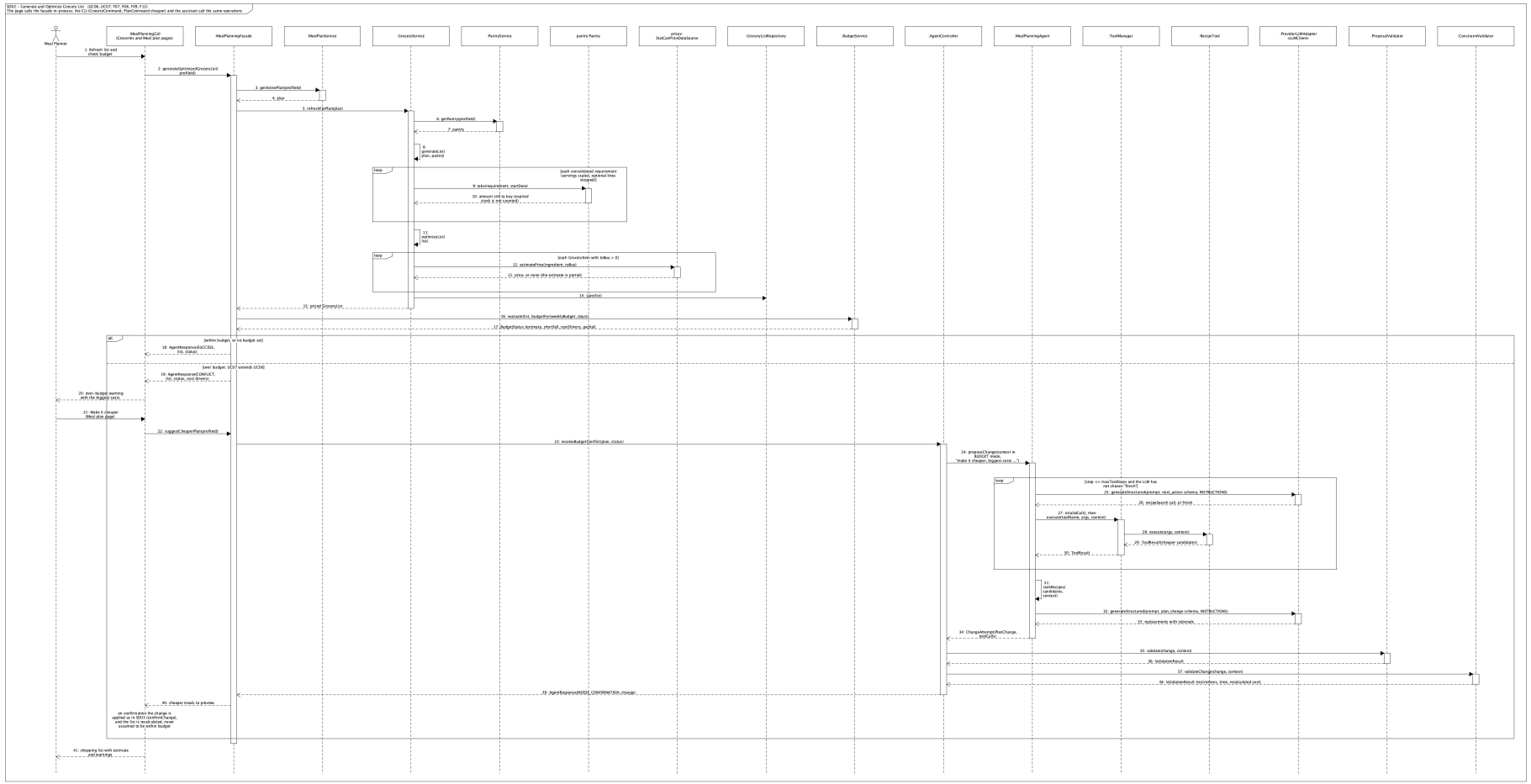
Use-case diagram (N01 and N02 added).



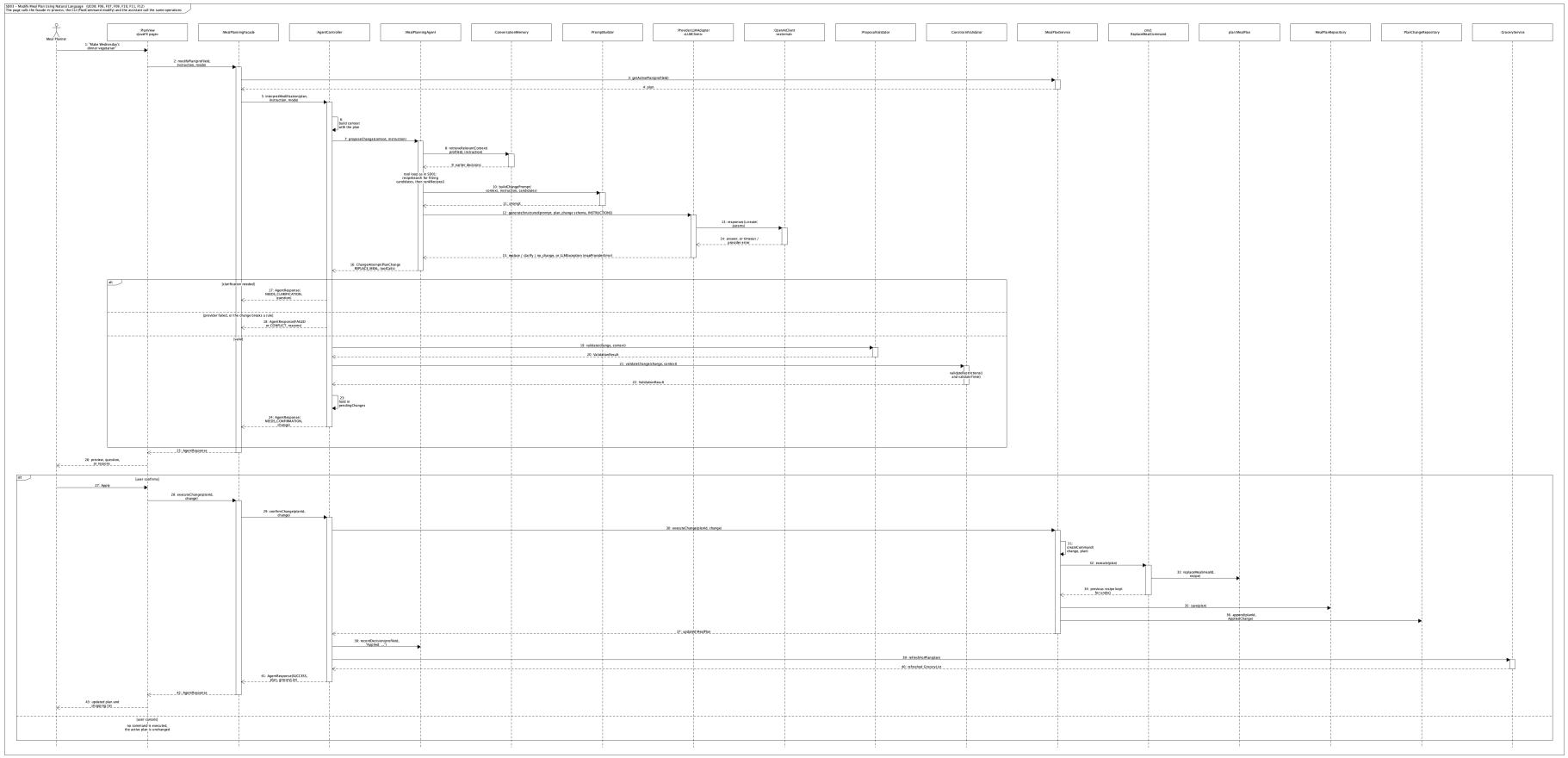
SD01: generate a weekly meal plan (automatic searches and portion sizing added).



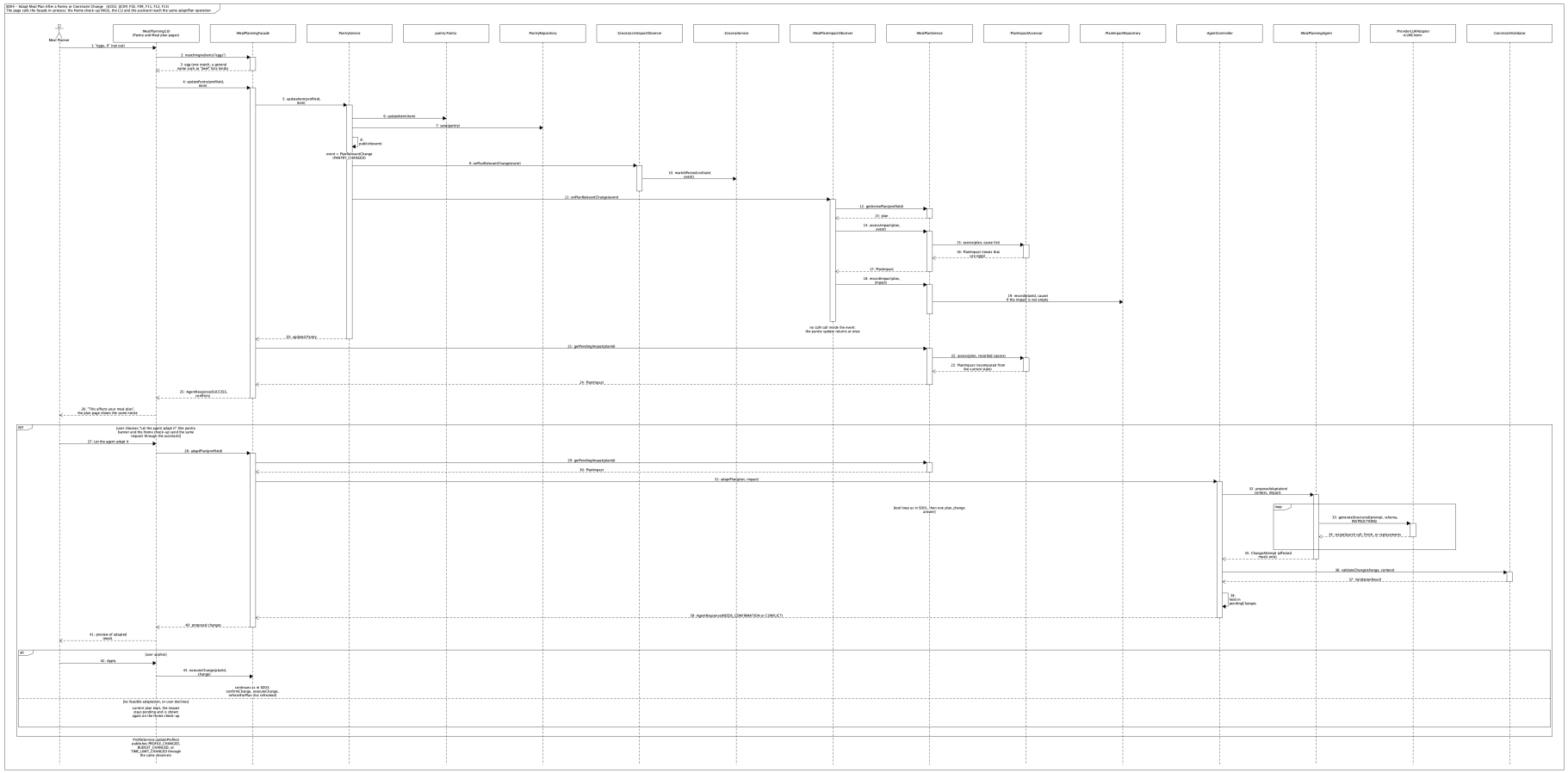
SD02: generate and optimize the grocery list.



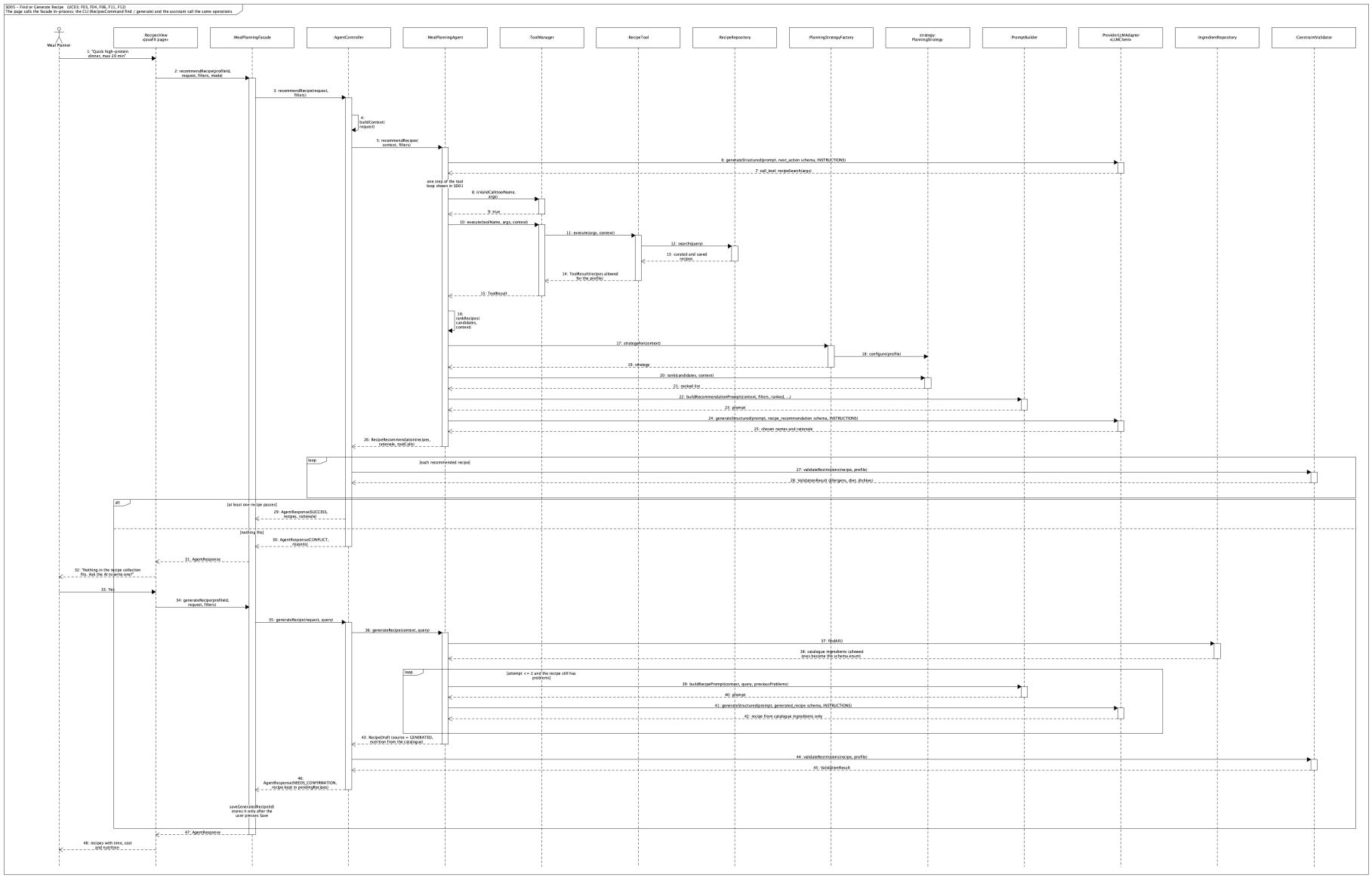
SD03: modify the plan in natural language.



SD04: adapt the plan after a pantry or constraint change.



SD05: find or generate a recipe (external recipe provider removed).



4. Changes and rationale

Each change is recorded as Original design -> Implemented change -> Reason -> Impact.

ID	Original design	Implemented change	Reason	Impact
C01	Python, FastAPI, React, Typer, SQLAlchemy, Pydantic, pytest	Java 21, Maven wrapper, JavaFX, picocli, JDBC, Jackson, JUnit 5. The Python code (commits up to `756701c`) was replaced layer by layer from Oct 30	The instructor required Java on Oct 30, 2026	Same architecture and class names. All layers rewritten; Stage 1 technology references updated
C02	GUI reaches the facade through `MealPlanningApi` (HTTP)	GUI and CLI call `MealPlanningFacade` in-process; no API layer	A JavaFX desktop app runs in the same process, so HTTP adds no value (D02 kept its intent: one shared boundary)	`MealPlanningApi` removed; `MealPlanningGUI` has no `apiBaseUrl`
C03	Facade with 11 operations	Additional read and helper operations: `getProfile`, `getPantry`, `pantryItem`, `removePantryItem`, `searchIngredients`, `findRecipe`, `recipeNutrition`, `recipeCost`, `browseRecipes`, `getActivePlan`, `dailyNutrition`, `changeHistory`, `pendingImpact`, `ensureProfile`, `localProfileId`	The interfaces must display state, and a local single-user app selects profiles by name	Facade remains the only entry point
C04	`LLMClient.generateStructured(prompt, schema)`	Adds an `instructions` parameter; schema is a `JsonSchema` class that builds strict schemas and validates answers	Fixed system guidance is separate from the prompt; every client, including the test client, guarantees schema-valid answers	`ScriptedLLMClient` added for tests (D08)
C05	`AgentTool.execute(args)`	`execute(args, context)`; tools also provide `description()` and `checkArguments()`	Tools need the profile and pantry to filter results; `ToolManager.isValidCall` needs per-tool argument checks	`ToolResult` gained a text `summary` used as the LLM observation; `ToolCall` records each step
C06	`recommendRecipes` returns `List<Recipe>`; `createPlanProposal` returns `PlanningProposal`	Records `RecipeRecommendation`, `PlanAttempt`, `ChangeAttempt`, `RecipeDraft` return results plus tool calls	The tool calls are shown to the user ("How I did it") and used in tests	`AgentResponse` gained `agentSteps`
C07	Tool loop arguments as an object	`next_action` schema carries arguments as a JSON string	OpenAI strict structured output cannot express free-form objects	`ToolManager` parses and checks them; invalid JSON is an observation
C08	`PromptBuilder` with five prompt methods	Prompts for tool selection, recommendation, planning, change, substitution, recipe and assistant; shared `INSTRUCTIONS`	Each agent operation needs its own context; every number in a prompt comes from deterministic services	Costs per serving and the budget appear in prompts (C17)
C09	`UserProfile` field validation not specified for allergens	Unknown allergy tags are rejected; `DietRules.knownAllergens`	An allergy that matches no catalogue tag would never be enforced (D04)	GUI uses checkboxes for diets and allergens
C10	Pantry requires an existing profile	`ensureProfile` creates a default profile on first pantry use	Users filled the pantry first and hit a database error	Found in user testing
C11	Recipe data from curated and external sources	30 curated seed recipes (`catalogue/recipes.json`) using only catalogue ingredients, loaded by `mealiq setup`	Every recipe must have complete USDA nutrition (D09)	`RecipeCatalogueLoader`; tests check every diet tag against the diet rules

ID	Original design	Implemented change	Reason	Impact
C12	`DatabaseSetup` not in the design	`mealiq setup` creates missing tables and loads ingredients and recipes; idempotent	A reproducible local database	13 tables in `db/schema.sql`
C13	`ExternalRecipeAdapter` / `RecipeDataSource` for an external recipe API	Not implemented; search covers curated and saved generated recipes	Time; external ingredients would need matching to the catalogue (D09) to keep nutrition exact. F04 covers gaps	F03 modified; the Adapter pattern is still implemented for the LLM and prices
C14	Planning strategies return `rank` only	`PlanningStrategy.score` public, `rank` default; `UNSCORED` for unmeasurable fits	Shared ordering rules; testable scores	Strategy pattern unchanged in role
C15	`PlanningStrategyFactory` as interface	Abstract class whose `strategyFor` creates and configures	The factory method keeps creation and configuration together	Factory Method unchanged in role
C16	`BudgetService.estimateCost` returns a Decimal	Returns `CostEstimate` (known cost, priced and unpriced counts); `estimateRecipeCost`, `evaluate`, `budgetFor`	Many ingredients have no public price; unknown prices must never be guessed	Costs shown as `$x+`
C17	Price source unspecified	`StatCanPriceDataSource` with 63 Statistics Canada prices, rebuilt by `scripts/build_prices.py`; units whose meaning differs (head vs leaf) left out	Real, citable public data	Budget ranking and checks work on known prices
C18	`ConstraintValidator` without dependencies	Takes `NutritionService` and `BudgetService`; `validatePlan`, `validateChange`, `validateTime`, `validateRestrictions`	Plan checks need nutrition and cost	Optional ingredients and dislikes give warnings only
C19	Cooking-time limit semantics open	The limit applies to each meal on that day	Matches `validateTime(meal, profile)`	Over-limit meals are violations
C20	Meal slots by meals per day not specified	`MealSlots`: 1 = dinner, 2 = lunch and dinner, 3 = plus breakfast, more = snacks; one serving per meal	Deterministic slot structure for validation	Limitation on calorie targets (Section 1)
C21	Plan variety not specified	Same recipe twice on one day is a violation; more than twice in a plan is a warning	A real run produced the same dish four times	Found in a real OpenAI run
C22	`MealPlanService.createFromProposal(proposal)`	`createFromProposal(request, proposal)` saves the plan as active and archives the previous one	The proposal does not carry the profile and dates	One active plan per profile
C23	`MealPlanService.history: Deque<PlanChangeCommand>` in memory	Applied changes are stored in table `plan_changes` (`AppliedChange`, `MealSwap`, `PlanChangeRepository`); undo rebuilds the commands	Each CLI command is a new process, so in-memory history would be lost	Undo works across sessions and interfaces
C24	Three commands	Adds `CompositeCommand` (macro) and `PlanChangeCommand.swaps()`	One confirmed change may touch several meals and must undo as a unit	Command pattern extended, not replaced
C25	Substitution edits a recipe	`SubstituteIngredientCommand.variant` creates a stored recipe variant (`RecipeSource.VARIANT`) hidden from search	Plans reference recipes by ID; the original recipe must stay unchanged for other meals and undo	New enum value

ID	Original design	Implemented change	Reason	Impact
C26	`MealPlanService.pendingImpacts` in memory	Impact causes stored in table `plan_impacts` (`ImpactCause`); `getPendingImpact` recomputes the impact from the current state	Survives between CLI commands; a fixed problem disappears and returns after an undo	New `PlanImpactAssessor`; `PlanImpact` gained `causes`
C27	Observers call `MealPlanService` and `GroceryService`	Same, plus `PlanImpactAssessor` for the deterministic checks; the grocery list becomes stale only on pantry changes	Profile changes do not change what must be bought	Observer pattern unchanged in role
C28	`AgentController` fields	Adds `pendingChanges`, `pendingRecipes`, `GroceryService`; methods `generateWeeklyPlan`, `recommendRecipe`, `generateRecipe`, `interpretModification`, `requestSubstitution`, `resolveBudgetConflict`, `adaptPlan`, `confirmChange`, `interpretRequest`	Confirmation of changes and generated recipes; budget alternatives	Matches SD01, SD03, SD04, SD05 flows
C29	`MealPlanningGUI` with show methods	JavaFX app with tabs; `showPlan`, `showChangePreview` in `PlanView`, `showGroceryList` in `GroceryView`; background worker for AI calls	JavaFX structure; the window must not freeze during AI calls	`GuiContext`, `Ui`, `MealIQDesktop` launcher
C30	Not in Stage 1	New feature N01, the Assistant: `MealPlanningAgent.interpretRequest`, `AssistantRequest` (12 actions), `MealPlanningFacade.ask/confirmUpdate`, `AssistantView` (first GUI tab), `mealiq ask`	User testing found the form-based GUI hard to use and not agent-like; one typed request can now drive every feature	Adds an AI routing step in front of existing use cases; all checks and confirmations unchanged
C31	`ConversationMemory` retrieval not specified	Keeps the newest entries per profile in memory; keyword match, else the most recent	Sufficient for follow-up requests in one session	Memory does not persist across CLI processes
C32	One serving per planned meal	New `PortionService` sizes each meal at 1 or 2 servings after the LLM chooses recipes, moving each day closer to its calorie and protein targets while known costs stay within budget	A real 7-day plan for 3000 kcal and 160 g protein could not get near the target with one serving per meal; portion sizing is arithmetic, which the LLM must not do (D01)	`AgentController.generateWeeklyPlan` calls it before validation; CLI and GUI show "x2" portions
C33	Candidates per meal type only	The planning prompt lists, for each time-limited day, only the candidates that fit its limit; quick candidates are always included; the LLM is told not to filter searches by pantry items; when its searches find too few recipes, the agent adds a broad search (and a time-limited one) automatically	In a real run the LLM added every pantry item as an `include` filter (2-4 candidates left), and placed 40-minute recipes on 20-minute days, so the plan was rejected twice	The same 7-day plan passed on the first attempt in a real run
C34	Pantry needs exact catalogue names	`PantryService.matchIngredients` maps everyday names: plurals ("eggs") to one ingredient, general names ("beef") to every kind; the GUI asks which kind, filters the list as you type, and hands unknown names to the agent; the assistant asks which kind instead of guessing; the CLI error lists the kinds	User testing: "beef" was rejected and the list did not narrow while typing	Only exact or plural names are saved without a question

ID	Original design	Implemented change	Reason	Impact
C35	Not in Stage 1	New feature N02: `MealPlanningFacade.checkup` and `Checkup` (today's meals, pantry and grocery status, suggestions), shown on the GUI Home page; accepting a suggestion sends its request to the agent	User testing: the app felt like a chatbot that waits to be asked; an agent should notice problems (plan no longer fits, expired items, over budget) and propose the fix	Detection is deterministic; every change still goes through the agent's preview and confirmation (D03)
C36	`MealPlanningGUI` (tabs, C29)	Sidebar navigation with a Home page and a shared stylesheet (`mealiq.css`); the Plan page shows day cards, a clear status after generating (saved automatically when every check passes, or not saved, with details folded away) and the change actions on one card	User testing: the plan screen was confusing (no visible save step, about 30 lines of output in a text box) and the look was not engaging	`GuiSnapshots` renders all seven pages

5. AI tools and collaboration log

AI tools and models

Tool or platform	Model	Used for	Tokens and cost
Claude Code (desktop app, Code tab)	Claude Opus 5.5 (`claude-opus-5-5`)	Java port of every layer; agent, tools, strategies, commands, observers, validators, CLI, GUI; tests; data scripts (USDA catalogue, Statistics Canada prices); real-run verification; this report	Claude Pro subscription (fixed monthly price; no per-token charge). Claude Code does not report a per-project token total; the main implementation session's context reached about 860,000 tokens before compaction
OpenAI API (runtime LLM of MealIQ)	`gpt-5.4-mini-2026-03-17`	The agent's tool selection, choices, explanations, recipe generation and request interpretation at run time, and the development test runs	Total from the OpenAI usage dashboard: to be added

Representative interactions

Task	Input from the team	AI contribution	Human contribution	Outcome
Switch to Java	Instructor requirement (Oct 30)	Proposed the stack (JavaFX, picocli, JDBC, OpenAI Java SDK) keeping the Stage 1 classes; ported layer by layer with tests	Decided to follow the requirement and keep the Stage 1 design	Commits `b22fc7f` to `8d96a77`
API key safety	First project with an API key	Explained `.env`, `.gitignore`, spending limits and key revocation; verified the ignore rule; ran one small real call without printing the key	Created the key and `.env` locally, set a spending limit	Model name verified by a real structured-output call
Commit cadence	Weekly minimum of 5 code commits	Grouped work into one meaningful step per commit, naming feature IDs	Decided when to commit and pushed	Every period above the minimum (Section 6)
Grocery prices	Budget features needed real prices	Proposed Statistics Canada table 18-10-0245-01, asked before downloading, built a reproducible script and left mismatched units unpriced	Approved the download	63 priced ingredients, citable
Agent quality from real runs	Real OpenAI runs during development	Found and fixed: unknown prices making all costs unknown, the same dish four times in a plan, candidate lists hiding vegetarian dinners, "a dozen eggs" stored as 144	Reviewed the runs and the fixes	Fixes C16, C21, and the assistant schema (C30)
GUI usability	Hands-on test with screenshots: database error on the pantry, free-text allergies, a dead end in recipe search, "doesn't feel like AI"	Fixed the defects; offered three directions and the cost of each	Chose fixes plus an Assistant tab	New feature N01, checkboxes, friendly errors
Plan failures and agent feel	Second hands-on test: a 7-day plan rejected twice, "beef" rejected, a confusing plan screen, a request for a sidebar and home page, and "an AI agent, not a chatbot"	Traced the failures in the agent steps to over-narrow searches, missing per-day time filtering and fixed portions; redesigned the GUI and added a proactive check-up	Reported the problems with screenshots and decided the order of work	C32-C36, new feature N02; the same plan passed on its first attempt

6. Development and verification summary

Weekly commit summary (Toronto time)

Period	Code commits	Commits
1: Oct 20-26	6 (Python implementation, replaced in Period 2)	37e87c6, 1ef239a, f6cd3e9, 90d685e, 267a167, 3a9b1af
2: Oct 27-Nov 2	8	756701c, b22fc7f, cd8f536, 91af3ae, 2258b50, 920fcb2, 8d96a77, fd07f1a
3: Nov 3-9	14	2941ad5, 569cddf, 7032343, 5ebc720, 56ec44e, 17faa1c, ae14036, 16795b1, 3f4d9b9, b0797c1, 06e2277, 9b91d05, b67e319, cfaddd3

Checks performed

- **Automated tests:** 369 JUnit 5 tests (one PostgreSQL test class is skipped unless a test database is configured). They cover domain values and units, repositories (as contract tests run on H2 and PostgreSQL), the catalogue and price data, every service and validator, the strategies, the commands, the agent loop driven by `ScriptedLLMClient` (valid, invalid and malformed tool calls, the step limit, filters the LLM cannot widen, fallbacks), the controller and facade flows, and the CLI commands end to end. The build compiles with `-Xlint:all` and no warnings.
- **Real runs:** every AI feature was run through the CLI against the real OpenAI API and PostgreSQL 16:
  - recipe recommendation
  - generation
  - 2- and 3-day plans (including budget mode with a rejected first attempt)
  - typed changes with confirmation and undo
  - substitution
  - adaptation after a tighter time limit and after running out of an ingredient
  - priced grocery lists
  - a full Assistant conversation from an empty database
  - a 7-day plan with 20-minute limits on Friday to Sunday, 3000 kcal and 160 g protein, which passed on its first attempt with portions sized up (C32, C33)
  - "I have 2 pounds of beef" through the Assistant, which asked whether it was ground beef or beef sirloin (C34)
- **GUI:** every page was rendered off-screen with test data (`GuiSnapshots`) and inspected, and the real app was launched against PostgreSQL without errors. Clicking through the pages by hand was done by the team, not by an automated test.

Important defects found and corrected

- An allergy typed as "peanuts" was saved but matched no catalogue tag, so it would never be enforced. Unknown allergens are now rejected (C09).
- One unknown price made whole recipe costs unknown, which disabled budget ranking. Costs are now lower bounds (C16).
- A real plan repeated one dish four times. Repeating a recipe within a day is now a violation (C21).
- The agent's change candidates were cut to the top 16 overall, hiding the vegetarian dinners a request needed. The agent now sees all results of its own searches.

- In-memory undo history and pending impacts did not survive between CLI commands. They are now persisted (C23, C26).
- Using the pantry before saving a profile caused a database error. A profile is now created automatically (C10).
- "A dozen eggs" was stored as 12 dozen, and "rice" was not matched. Assistant pantry lines are now limited to catalogue names, with explicit amount rules (C30).
- A recipe search with no matches ended without a next step. Generation is now offered.
- A 7-day plan was rejected twice: searches filtered by pantry items left 2-4 candidates, and 40-minute recipes were placed on 20-minute days (C33). One serving per meal could not reach 3000 kcal (C32).
- "beef" was rejected in the pantry because only exact catalogue names were accepted (C34).

Remaining limitations

See Section 1. In addition, `ConversationMemory` is per process, so a follow-up such as "ground beef" to the agent's question works in the GUI session but not across separate CLI commands.

7. Demonstration index

Feature	Interface	Video segment
F01 User goals and preferences	GUI and CLI	pending
F02 Pantry inventory management	GUI and CLI	pending
F03 Recipe search and recommendation	GUI and CLI	pending
F04 AI recipe generation	GUI and CLI	pending
F05 Weekly meal-plan generation	GUI and CLI	pending
F06 Nutrition analysis	GUI and CLI	pending
F07 Grocery-list generation	GUI and CLI	pending
F08 Grocery-list optimization	GUI and CLI	pending
F09 Ingredient substitution	GUI and CLI	pending
F10 Natural-language plan modification	GUI and CLI	pending
F11 Cooking-time-aware planning	GUI and CLI	pending
F12 Budget-aware planning	GUI and CLI	pending
F13 Dynamic plan adaptation	GUI and CLI	pending
N01 Assistant	GUI and CLI	pending
N02 Agent check-up	GUI	pending